chatbot.js - Retrieval Chatbot for Student Task & Follow-up Tracker

```
/**
 * chatbot.js - Retrieval-based chatbot for the Student Task & Follow-up Tracker
 *
 * How it works (no API, no server, runs fully in the browser):
 *  1. Tasks are loaded from localStorage (key: TASK_STORAGE_KEY) or from sample data.
 *  2. Each task is turned into a searchable "document" and indexed with TF-IDF.
 *  3. The user's question is scored against every task (cosine similarity).
 *  4. Special intents (overdue, due today, pending, completed, follow-ups) are
 *     handled with simple filters before falling back to similarity search.
 *
 * Usage: add before </body> in index.html
 *   <script src="chatbot.js"></script>
 *
 * Task shape expected:
 *   { id, title, description, dueDate: "YYYY-MM-DD", status: "pending"|"completed", followUp:
    "text" }
 * Change TASK_STORAGE_KEY / field names below to match your app.
 */

const TASK_STORAGE_KEY = "tasks";

const SAMPLE_TASKS = [
  { id: 1, title: "Submit DSP assignment", description: "Image restoration problems from unit 4",
      dueDate: "2026-11-06", status: "pending", followUp: "Ask staff about the format" },
  { id: 2, title: "Push project to GitHub", description: "Commit tracker code and create
      repository", dueDate: "2026-11-05", status: "pending", followUp: "Add README" },
  { id: 3, title: "Revise sequential logic", description: "Flip-flops, counters, registers",
      dueDate: "2026-10-30", status: "completed", followUp: "" },
];

/* ---------- Data ---------- */

function loadTasks() {
  try {
    const raw = localStorage.getItem(TASK_STORAGE_KEY);
    const parsed = raw ? JSON.parse(raw) : null;
    if (Array.isArray(parsed) && parsed.length) return parsed;
  } catch (e) {
    console.warn("Could not read tasks from localStorage:", e);
  }
  return SAMPLE_TASKS;
}

/* ---------- Retrieval engine (TF-IDF + cosine similarity) ---------- */

const STOPWORDS = new Set(
  "a an the is are was were be to of in on at for and or my me i do does did what which who when
      where how show tell give list any all about with from this that it".split(" ")
);

function tokenize(text) {
  return (text || "")
    .toLowerCase()
    .replace(/[^a-z0-9\s]/g, " ")
    .split(/\s+/)
    .filter((w) => w && !STOPWORDS.has(w))
    .map((w) => (w.length > 3 && w.endsWith("s") ? w.slice(0, -1) : w)); // crude stemming
}

class TaskRetriever {
  constructor(tasks) {
    this.tasks = tasks;
    this.docs = tasks.map((t) =>
      tokenize([t.title, t.description, t.followUp, t.status, t.dueDate].join(" "))
    );
    this.idf = this._buildIdf();
    this.vectors = this.docs.map((d) => this._vectorize(d));
  }

  _buildIdf() {
    const df = {};
    this.docs.forEach((d) => new Set(d).forEach((w) => (df[w] = (df[w] || 0) + 1)));
    const n = this.docs.length;
    const idf = {};
```

```
    for (const w in df) idf[w] = Math.log((n + 1) / (df[w] + 1)) + 1;
    return idf;
  }

  _vectorize(tokens) {
    const tf = {};
    tokens.forEach((w) => (tf[w] = (tf[w] || 0) + 1));
    const vec = {};
    for (const w in tf) if (this.idf[w]) vec[w] = tf[w] * this.idf[w];
    return vec;
  }

  _cosine(a, b) {
    let dot = 0, na = 0, nb = 0;
    for (const w in a) { na += a[w] ** 2; if (b[w]) dot += a[w] * b[w]; }
    for (const w in b) nb += b[w] ** 2;
    return na && nb ? dot / (Math.sqrt(na) * Math.sqrt(nb)) : 0;
  }

  search(query, topK = 3, minScore = 0.1) {
    const q = this._vectorize(tokenize(query));
    return this.tasks
      .map((task, i) => ({ task, score: this._cosine(q, this.vectors[i]) }))
      .filter((r) => r.score >= minScore)
      .sort((a, b) => b.score - a.score)
      .slice(0, topK);
  }
}

/* ---------- Answer generation ---------- */

const todayStr = () => new Date().toISOString().slice(0, 10);

function fmt(task) {
  const status = task.status === "completed" ? "[done]" : "[pending]";
  const due = task.dueDate ? ` (due ${task.dueDate})` : "";
  const fu = task.followUp ? `\n   Follow-up: ${task.followUp}` : "";
  return `${status} ${task.title}${due}${fu}`;
}

function listReply(heading, tasks) {
  return tasks.length ? `${heading}\n${tasks.map(fmt).join("\n")}` : `${heading}\nNothing found.`;
}

function getAnswer(question, tasks, retriever) {
  const q = question.toLowerCase().trim();
  const today = todayStr();

  if (/^(hi|hello|hey)\b/.test(q))
    return "Hi! Ask me about your tasks - e.g. "what is pending?", "what is overdue?", "what is
        due today?" or "find GitHub task".";

  if (/help|what can you do/.test(q))
    return "I can show pending, completed, overdue, due-today tasks and follow-ups, or search
        tasks by any keyword.";

  if (/overdue|late|missed/.test(q))
    return listReply("Overdue tasks:", tasks.filter((t) => t.status !== "completed" && t.dueDate
        && t.dueDate < today));

  if (/today/.test(q))
    return listReply("Due today:", tasks.filter((t) => t.status !== "completed" && t.dueDate ===
        today));

  if (/follow.?up/.test(q))
    return listReply("Follow-ups:", tasks.filter((t) => t.followUp && t.followUp.trim()));

  if (/pending|incomplete|to ?do|left|remaining/.test(q))
    return listReply("Pending tasks:", tasks.filter((t) => t.status !== "completed"));

  if (/completed|done|finished/.test(q))
    return listReply("Completed tasks:", tasks.filter((t) => t.status === "completed"));

  if (/how many|count|total/.test(q)) {
    const done = tasks.filter((t) => t.status === "completed").length;
    return `You have ${tasks.length} tasks: ${done} completed, ${tasks.length - done} pending.`;
  }
  // Fallback: semantic-ish retrieval
```

```
  const results = retriever.search(question);
  if (!results.length)
    return "I couldn't find a matching task. Try different keywords, or ask "what is pending?".";
  return "Best matches:\n" + results.map((r) => fmt(r.task)).join("\n");
}

/* ---------- Chat widget UI (injected automatically) ---------- */

function mountChatbot() {
  const style = document.createElement("style");
  style.textContent = `
    #cb-toggle{position:fixed;bottom:20px;right:20px;width:56px;height:56px;border-
        radius:50%;border:none;background:#4f46e5;color:#fff;font-size:26px;cursor:pointer;box-
        shadow:0 4px 12px rgba(0,0,0,.25);z-index:9999}
    #cb-box{position:fixed;bottom:88px;right:20px;width:320px;max-width:calc(100vw -
        40px);height:420px;background:#fff;border:1px solid #ddd;border-
        radius:12px;display:none;flex-direction:column;overflow:hidden;box-shadow:0 8px 24px
        rgba(0,0,0,.2);font-family:system-ui,sans-serif;z-index:9999}
    #cb-head{background:#4f46e5;color:#fff;padding:10px 14px;font-weight:600}
    #cb-msgs{flex:1;overflow-y:auto;padding:10px;display:flex;flex-
        direction:column;gap:8px;background:#f8f8fc}
    .cb-m{padding:8px 10px;border-radius:10px;max-width:85%;white-space:pre-wrap;font-
        size:14px;line-height:1.35}
    .cb-u{align-self:flex-end;background:#4f46e5;color:#fff}
    .cb-b{align-self:flex-start;background:#fff;border:1px solid #e3e3ee;color:#222}
    #cb-form{display:flex;border-top:1px solid #ddd}
    #cb-in{flex:1;border:none;padding:10px;font-size:14px;outline:none}
    #cb-send{border:none;background:#4f46e5;color:#fff;padding:0 16px;cursor:pointer}
  `;
  document.head.appendChild(style);

  const toggle = document.createElement("button");
  toggle.id = "cb-toggle";
  toggle.textContent = "(chat)";
  toggle.setAttribute("aria-label", "Open task chatbot");

  const box = document.createElement("div");
  box.id = "cb-box";
  box.innerHTML = `
    <div id="cb-head">Task Assistant</div>
    <div id="cb-msgs"></div>
    <div id="cb-form"><input id="cb-in" placeholder="Ask about your tasks..." /><button id="cb-
        send">Send</button></div>`;

  document.body.append(toggle, box);

  const msgs = box.querySelector("#cb-msgs");
  const input = box.querySelector("#cb-in");

  const add = (text, who) => {
    const el = document.createElement("div");
    el.className = "cb-m " + (who === "u" ? "cb-u" : "cb-b");
    el.textContent = text;
    msgs.appendChild(el);
    msgs.scrollTop = msgs.scrollHeight;
  };

  const send = () => {
    const text = input.value.trim();
    if (!text) return;
    add(text, "u");
    input.value = "";
    // Reload tasks each time so new/edited tasks are always included
    const tasks = loadTasks();
    const retriever = new TaskRetriever(tasks);
    add(getAnswer(text, tasks, retriever), "b");
  };

  box.querySelector("#cb-send").addEventListener("click", send);
  input.addEventListener("keydown", (e) => e.key === "Enter" && send());
  toggle.addEventListener("click", () => {
    const open = box.style.display === "flex";
    box.style.display = open ? "none" : "flex";
    if (!open && !msgs.children.length) add("Hi! Ask me about your tasks. Type "help" to see what
        I can do.", "b");
  });
}
if (typeof document !== "undefined") {
```

```
  document.addEventListener("DOMContentLoaded", mountChatbot);
}
```